Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-211БВ-25

Студент: Шеменеева А. Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип float. Количество чисел может быть произвольным.

Использованные системные вызовы:

- pipe() - Создает однонаправленный канал связи (pipe) для передачи данных между процессами. В коде создаются два канала: client_to_server (для передачи команд от родителя к потомку) и server_to_client (для передачи результатов обратно).
- fork() - Создает новый дочерний процесс. Родительский процесс продолжает выполнение ветки default, а дочерний - ветки case 0.
- readlink() - Читает содержимое символической ссылки /proc/self/exe, чтобы получить абсолютный путь к исполняемому файлу текущей программы. Это необходимо для корректного поиска и запуска программы сервера из той же директории.
- dup2() - Дублирует файловые дескрипторы. Используется для перенаправления стандартного ввода на чтение из канала client_to_server и стандартного вывода на запись в канал server_to_client.
- execv() - Заменяет текущий образ процесса (дочернего) на новый исполняемый файл (программу сервера).
- wait() - Приостанавливает выполнение родительского процесса до тех пор, пока дочерний процесс не завершится.

Запуск родительского процесса

883 execve("./parent", ["./parent", "output.txt"], 0x7ffcb3d47d10 /* 26 vars */) = 0

Создание каналов

883 pipe2([3, 4], 0) = 0

883 pipe2([5, 6], 0) = 0

Форк дочернего процесса

883 clone(child_stack=NULL,flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7fdfd89bba10) = 884

Создан дочерний процесс

883 <... getpid resumed>) = 883

884 <... getpid resumed>) = 884

Оба процесса вывод инфармационое сообщение

883 write(2, "Parent process: successfully spa"..., 43 <unfinished ...>

884 write(2, "Child process started\n", 22 <unfinished ...>

Закрытие ненужных концов

883 close(3) # родитель закрывает read-end pipe1

884 close(4) # ребёнок закрывает write-end pipe1

883 close(6) # родитель закрывает write-end pipe2

884 close(5) # ребёнок закрывает read-end pipe2

Родитель читает stdin

883 read(0, <unfinished ...>

Ребенок подменяте stdio

884 dup2(3, 0) = 0

884 close(3) = 0

884 dup2(6, 1) = 1

884 close(6) = 0

Загрузка sss

brk(NULL) = 0x6285a3290000

884 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7e4d27879000

884 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)

884 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

884 fstat(3, {st_mode=S_IFREG|0644, st_size=21267, ...}) = 0

884 mmap(NULL, 21267, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7e4d27873000

884 close(3) = 0

884 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

884 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832

884 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784

884 fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0

884 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784

884 mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7e4d27600000

884 mmap(0x7e4d27628000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7e4d27628000

884 mmap(0x7e4d277b0000, 323584, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7e4d277b0000

884 mmap(0x7e4d277ff000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7e4d277ff000

884 mmap(0x7e4d27805000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7e4d27805000

884 close(3) = 0

884 mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7e4d27870000

884 arch_prctl(ARCH_SET_FS, 0x7e4d27870740) = 0

884 set_tid_address(0x7e4d27870a10) = 884

884 set_robust_list(0x7e4d27870a20, 24) = 0

884 rseq(0x7e4d27871060, 0x20, 0, 0x53053053) = 0

884 mprotect(0x7e4d277ff000, 16384, PROT_READ) = 0

884 mprotect(0x628569f57000, 4096, PROT_READ) = 0

884 mprotect(0x7e4d278b1000, 8192, PROT_READ) = 0

884 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0

884 munmap(0x7e4d27873000, 21267) = 0

Завершение

883 close(4) = 0

884 <... read resumed>"", 4095) = 0

883 close(5 <unfinished ...>

884 write(3, "\0", 1 <unfinished ...>

883 <... close resumed>) = 0

884 <... write resumed>) = 1

883 wait4(-1, <unfinished ...>

884 close(3) = 0

884 exit_group(0) = ?

884 +++ exited with 0 +++

Код программы

client.c

```
#include <stdint.h>
#include <stdlib.h>
#include <unistd.h>
#include <ctype.h>
#include <sys/wait.h>
#include <string.h>



static char SERVER_PROGRAM_NAME[] = "sss";

int main(int argc, char *argv[]) {
    if (argc == 1) {
        char msg [1024];
        uint32_t len = snprintf(msg, sizeof(msg) - 1, "usage: %s filename\n", argv[0]);
        write(STDERR_FILENO, msg, len);
        exit(EXIT_SUCCESS);
    }

    char progpath[1024];
    {
        ssize_t len = readlink("/proc/self/exe", progpath, sizeof(progpath) - 1);

        if (len == -1) {
            const char msg[] = "error: failed to read full program path\n";
            write(STDERR_FILENO, msg, sizeof(msg));
            exit(EXIT_FAILURE);
        }

        while (progpath[len] != '/')
            --len;

        progpath[len] = '\0';

    }

    int client_to_server[2];
    if(pipe(client_to_server) == -1) {
        const char msg[] = "error: failed to create pipe\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(EXIT_FAILURE);
    }

    int server_to_client[2];
    if(pipe(server_to_client) == -1) {
```

```
        const char msg[] = "error: failed to create pipe\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(EXIT_FAILURE);
    }

    const pid_t child = fork();

    switch (child) {
        case -1: {
            const char msg[] = "error: failed to spawn new process\n";
            write(STDERR_FILENO, msg, sizeof(msg));
            exit(EXIT_FAILURE);
        } break;

        case 0: {
            {
                pid_t pid = getpid();

                const char msg[] = "Child process started\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
            }

            close(client_to_server[1]);
            close(server_to_client[0]);

            dup2(client_to_server[0], STDIN_FILENO);

            close(client_to_server[0]);

            dup2(server_to_client[1], STDOUT_FILENO);
            close(server_to_client[1]);

            {
                char path[1024];
                strcpy(path, progpath);
                strcat(path, "/");
                strcat(path, SERVER_PROGRAM_NAME);
                // snprintf(path, sizeof(path) - 1, "%s/%s", progpath,
SERVER_PROGRAM_NAME);

                char *const args[] = {SERVER_PROGRAM_NAME, argv[1], NULL};

                int32_t status = execv(path, args);

                if (status == -1) {
                    const char msg[] = "error: failed to exec into new exectuable
image\n";
                    write(STDERR_FILENO, msg, sizeof(msg));
```

```
                    exit(EXIT_FAILURE);
                }
            }
        }break;

        default: {
            {
                pid_t pid = getpid();

                const char msg[] = "Parent process: successfully spawned child\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
            }

            close(client_to_server[0]);
            close(server_to_client[1]);

            char buf[4096];
            ssize_t bytes;

            while (bytes = read(STDIN_FILENO, buf, sizeof(buf))) {
if (bytes < 0) {
const char msg[] = "error: failed to read from stdin\n";
write(STDERR_FILENO, msg, sizeof(msg));
exit(EXIT_FAILURE);
} else if (buf[0] == '\n') {
break;
}

write(client_to_server[1], buf, bytes);

bytes = read(server_to_client[0], buf, sizeof(buf));
write(STDOUT_FILENO, buf, bytes);
}

close(client_to_server[1]);
close(server_to_client[0]);

wait(NULL);
} break;
    }
}
```

server.c

```
#include <stdint.h>

#include <stdlib.h>
```

```
     #include <unistd.h>

     #include <ctype.h>

     #include <fcntl.h>

     #include <float.h>

     #include <errno.h>

     #include <string.h>

     void myFun(const char *str, char * buf) { float num = 0.0f; char *end; const char *p =
str;

     while(*p != '\0') {
    while (isspace((unsigned char)*p)) {
        p++;
    }
    if (*p == '\0') {
        break;
    }


    errno = 0;
 float tmp = strtof(p, &end);


    if (end == p) {
        const char msg[] = "error: failed to read number\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(EXIT_FAILURE);
    }


    if (errno == ERANGE) {
        if (tmp >= FLT_MAX || tmp <= -FLT_MAX){
            const char msg[] = "error: overflow\n";
            write(STDERR_FILENO, msg, sizeof(msg));
        } else{
            const char msg[] = "error: number is too small\n";
            write(STDERR_FILENO, msg, sizeof(msg));
        }
        exit(EXIT_FAILURE);


    }
    num += tmp;
    p = end;
}

int i = 0;
if (num < 0) {
        buf[i++] = '-';
        num = -num;
    }



    int whole = (int)num;
    int por = 1;
```

```
    while (whole / por >= 10) {
        por *= 10;
    }

    while (por > 0)
    {
        int dig = whole / por;
        buf[i++] = '0' + dig;
        whole %= por;
        por /=10;
    }



    buf[i++] = '.';

    float fract = num - (int)num;
    for (int j = 0; j < 6; j++) {
        fract *= 10;
        int dig = (int)fract;
        buf[i++] = '0' + dig;
        fract -= dig;
    }

    buf[i] = '\0';



     }

     int main(int argc, char *argv[]) { char buf[4096]; ssize_t bytes;

     pid_t pid = getpid();

int32_t file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC | O_APPEND, 0660);

if (file == -1) {
    const char msg[] = "error: failed to open requested file\n";
    write(STDERR_FILENO, msg, sizeof(msg));
    exit(EXIT_FAILURE);
}

while ((bytes = read(STDIN_FILENO, buf, sizeof(buf) - 1)) > 0) {
    buf[bytes] = '\0';
    if (bytes < 0) {
        const char msg[] = "error: failed to read from stdin\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(EXIT_FAILURE);
    }

    // while ((n = getline))
    char newbuf [33];
    myFun(buf, newbuf);

    size_t len = strlen(newbuf);
```

```
    {

        int32_t written = write(file, newbuf, len);
        if (written != (ssize_t)len) {
            const char msg[] = "error: failed to write to file\n";
            write(STDERR_FILENO, msg, sizeof(msg));
            exit(EXIT_FAILURE);
        }

        {
            const char msg[] = "Server received: ";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
        }

        written = write(STDOUT_FILENO, newbuf, len);
        if (written != (ssize_t)len) {
            const char msg[] = "error: failed to echo\n";
            write(STDERR_FILENO, msg, sizeof(msg));
            exit(EXIT_FAILURE);
        }
    }

}

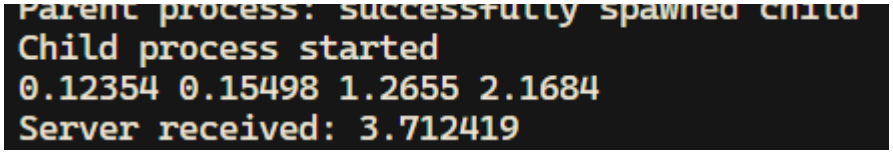
if (bytes == 0) {
    const char term = '\0';
    write(file, &term, sizeof(term));
}

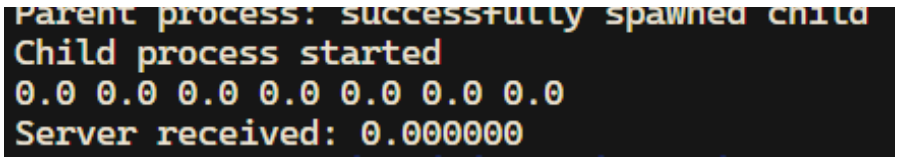
close(file);

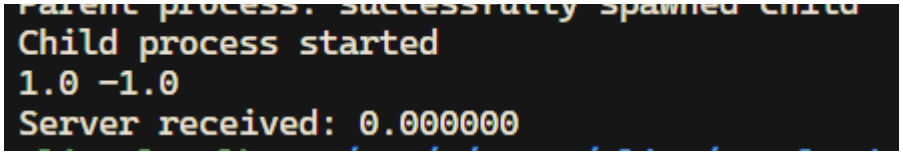

     }
```

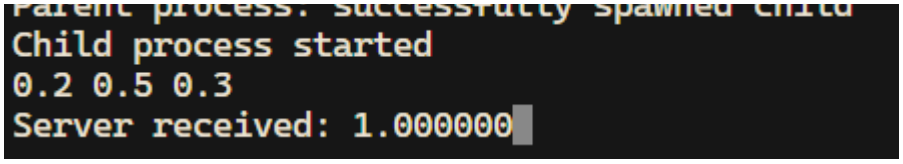
Протокол работы программы

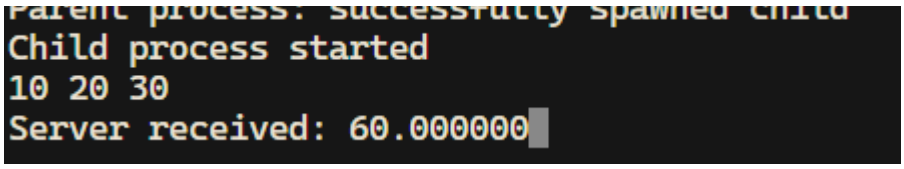
Тестирование:

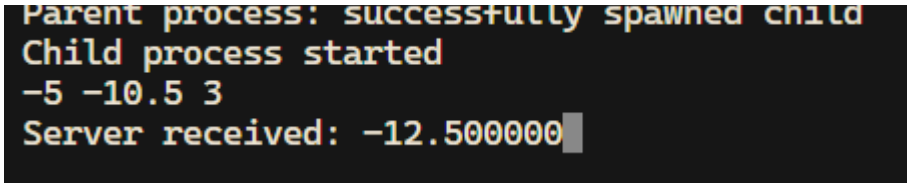












Вывод

В ходе выполнения лабораторной работы была успешно реализована схема межпроцессного взаимодействия через неименованные каналы (pipe) с использованием системных вызовов fork, execv и dup2. Основной проблемой была возникла при ручной реализации форматирования чисел с плавающей точкой, так как стандартные функции вывода могли вести себя непредсказуемо при работе с перенаправленными потоками. В целом, лабораторная позволила глубже понять механизмы работы операционной системы и управления процессами.